

CS-338 Intro to the Theory of Computation

- Lecture #3

Regular Expressions

- Prof Pietro S. Oliveto
Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)
- `olivetop@sustech.edu.cn`
<https://faculty.sustech.edu.cn/olivetop>
- Office: Room 113

Overview

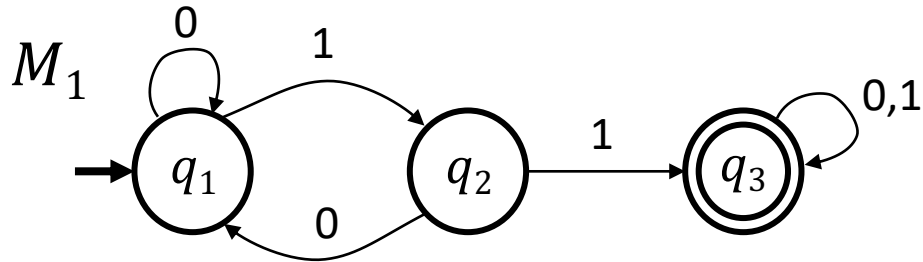
Last time:

- Nondeterminism
- NFA $\rightarrow$ DFA
- Closure under $\circ$ and $*$

Today: (Sipser §1.3)

- Regular expressions
- Regular expressions $\rightarrow$ finite automata
- Finite automata $\rightarrow$ regular expressions

Finite Automata (Recap)



States: $q_1 q_2 q_3$

Transitions: $\xrightarrow{1}$

Start state: $\rightarrow \bigcirc$

Accept states: $\bigcirc\bigcirc$

Input: finite string

Output: Accept or Reject

Computation process: Begin at start state, read input symbols, follow corresponding transitions, Accept if end with accept state, Reject if not.

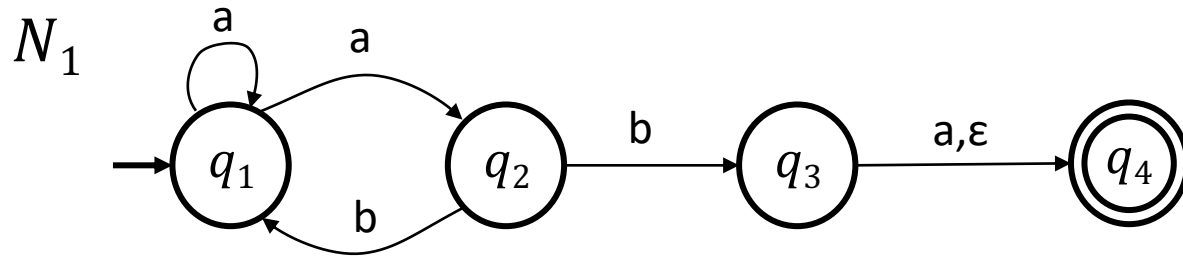
Examples: 01101 $\rightarrow$ Accept
00101 $\rightarrow$ Reject

M_1 accepts exactly those strings in A where
 $A = \{w \mid w \text{ contains substring } 11\}.$

Say that A is the language of M_1 and that M_1 recognizes A and that $A = L(M_1).$

A language is **regular** if some finite automaton recognises it

NFA (recap)



Defn: A nondeterministic finite automaton (NFA)

N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

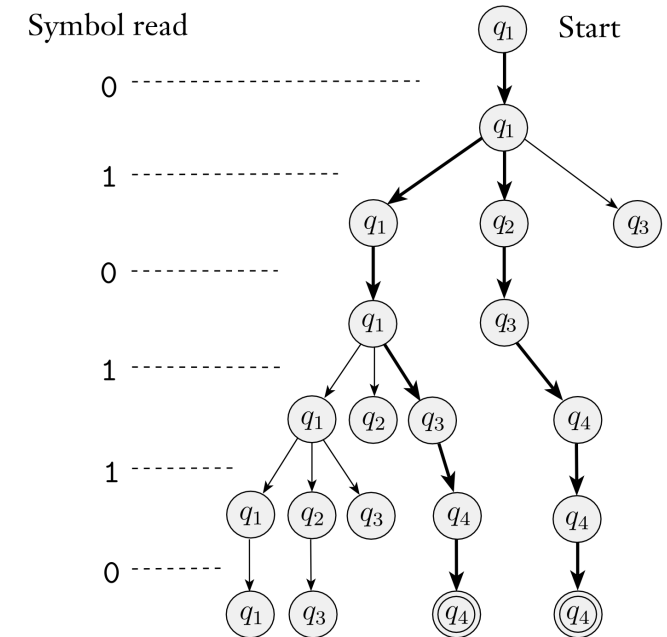
states
alphabet
transition function
start state
accept states

- all same as before except δ
- $\delta: Q \times \Sigma_\epsilon \rightarrow \mathcal{P}(Q) = \{R \mid R \subseteq Q\}$

$\Sigma \cup \{\epsilon\}$ power set

- In the N_1 example: $\delta(q_1, a) = \{q_1, q_2\}$
 $\delta(q_1, b) = \emptyset$

(A power set $\mathcal{P}(Q)$ is the collection of all possible subsets of Q)



Ways to think about nondeterminism:

Computational: Fork new parallel thread and accept if any thread leads to an accept state.

Mathematical: Tree with branches.

Accept if any branch leads to an accept state.

Magical: Guess at each nondeterministic step which way to go. Machine always makes the right guess that leads to accepting, if possible.

Converting NFAs to DFAs (Recap)

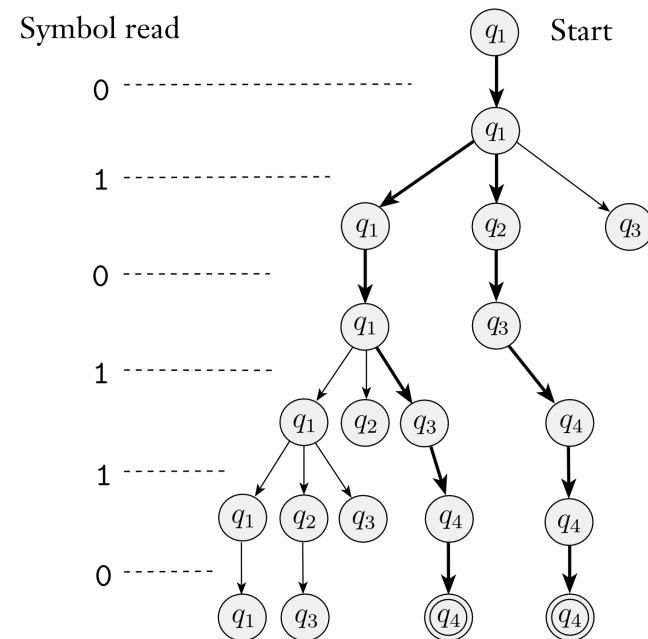
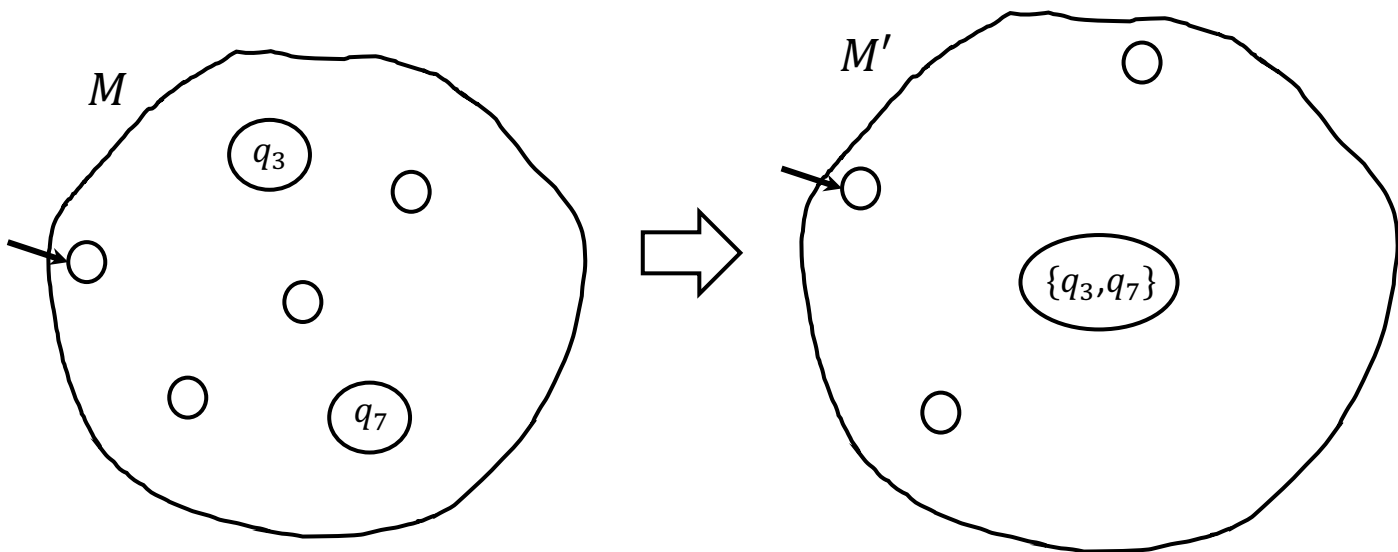
Theorem: If an NFA recognizes A then A is regular

Proof Idea: Let NFA $M = (Q, \Sigma, \delta, q_0, F)$ recognize A

Construct DFA $M' = (Q', \Sigma, \delta', q'_0, F')$ that recognizes A

(Ignore the ε -transitions, can easily modify to handle them)

IDEA: DFA M' keeps track of the subset of possible states in NFA M .



Construction of M' :

$$Q' = \mathcal{P}(Q)$$

$$\delta'(R, a) = \{q \mid q \in \delta(r, a) \text{ for some } r \in R\}$$

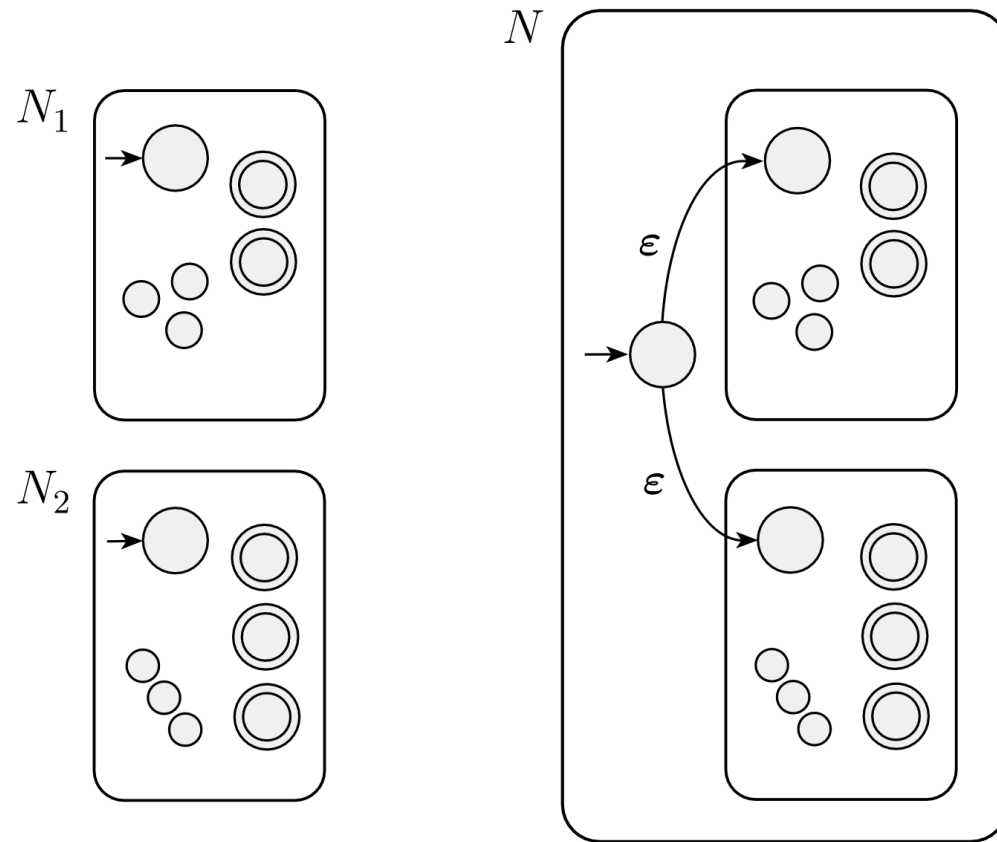
$$\overbrace{R \in Q'}^{\uparrow}. \quad \delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$$

Σ

$$q'_0 = \{q_0\}$$

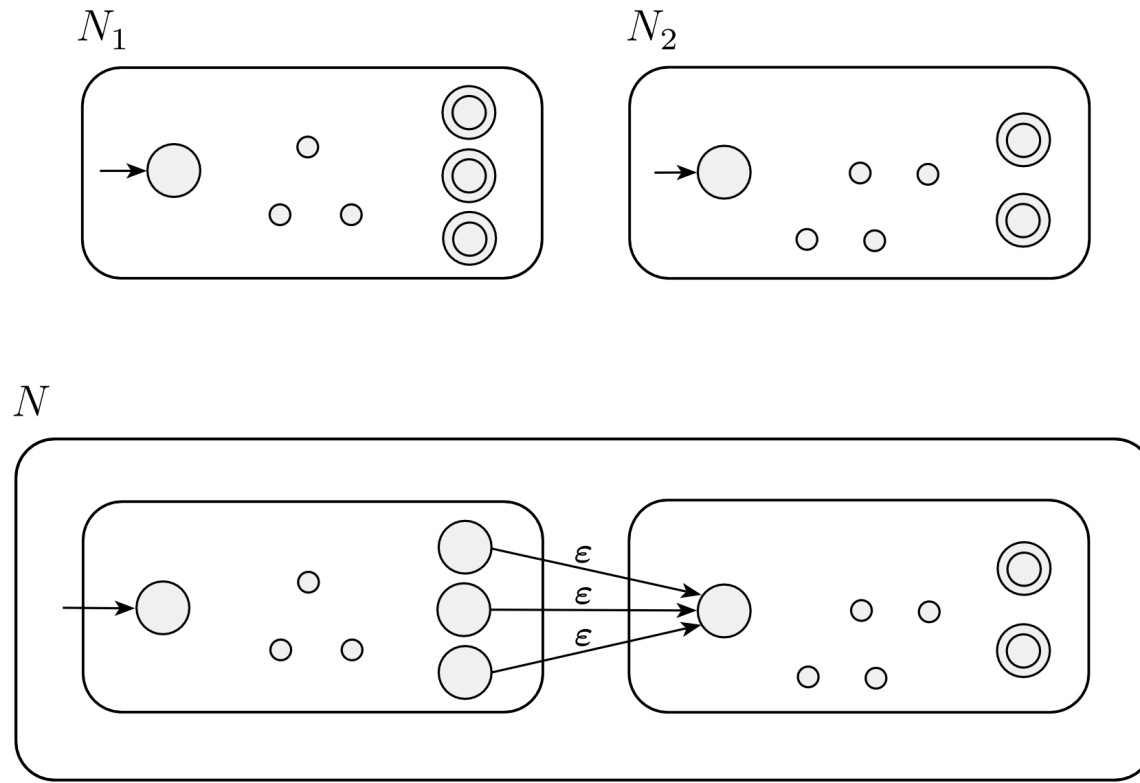
$$F' = \{R \in Q' \mid R \text{ intersects } F\}$$

Closure under Union (Recap)



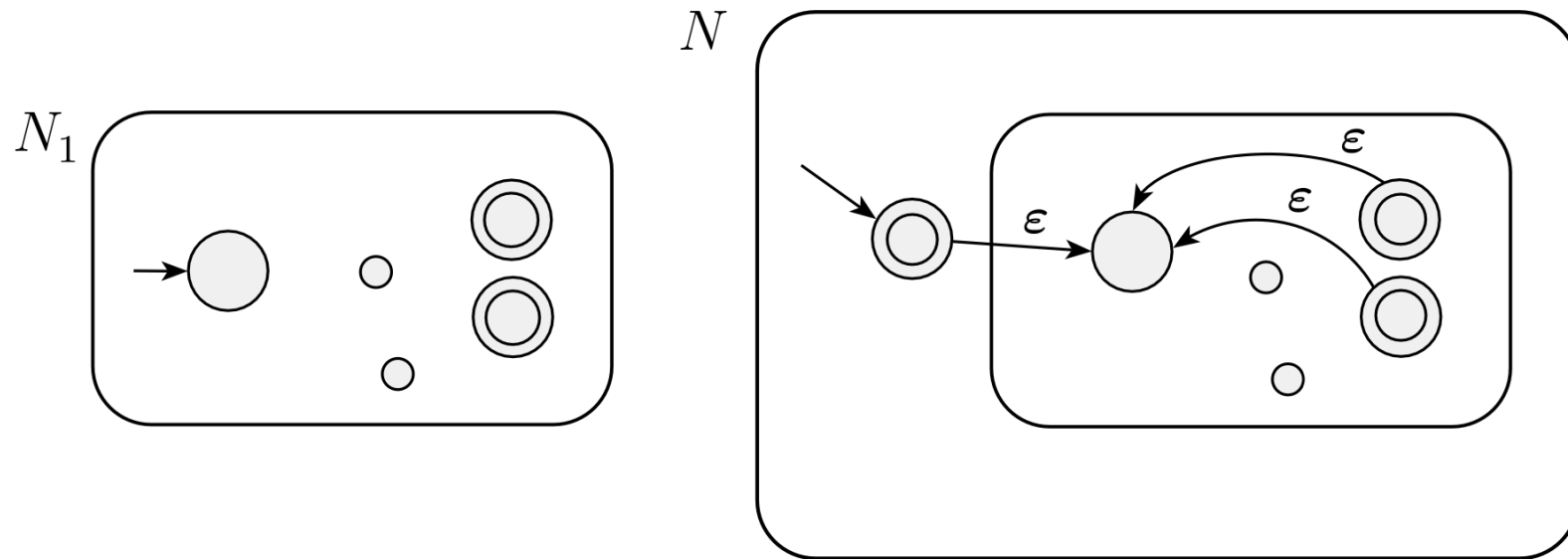
The new machine non-deterministically guesses which of the two machines accepts the input!

Closure under Concatenation (Recap)



We can think of new machine non-deterministically guessing where to split the input!

Closure under Star (Recap)



Regular Expressions

- What is computable (recognisable) by a finite state automaton?
- In arithmetic we use operations (eg., + and $\times$) to build expressions: eg., $(5+3) \times 4$
- Similarly we can use the regular operations ($\cup$, $\circ$, $*$) to describe languages: eg., $(0 \cup 1)0^*$
- The value of the arithmetic expression is a number i.e., 32. The value of the **regular expression** is a language: the language consisting of all strings starting with either a 0 or a 1 followed by any number of 0s.
 1. $\{0\}$ and $\{1\}$ are both languages
 2. $(0 \cup 1)$ [which means $\{0\} \cup \{1\}$] $\rightarrow$ the union of the two languages which is a language
 3. 0^* means $\{0\}^*$ $\rightarrow$ which is a language
 4. $(0 \cup 1)0^*$ is $\{(0 \cup 1)\} \circ \{0\}^*$ the concatenation of two languages which is a language
- Regular expressions play an important role in computer science applications involving text
- Utilities such as GREP and AWK in UNIX, programming languages such as Perl and text editors all provide mechanisms for the description of patterns using regular expressions
- They are very useful in compilers (to analyze tokens described by reg. expression and check syntactic correctness)

Regular Expressions: Formal Definition

Say that R is a *regular expression* if R is

1. a for some a in the alphabet Σ , $\{a\}$
2. ϵ , $\{\epsilon\} \rightarrow$ language containing one string
3. $\emptyset$, Empty language $\rightarrow L$ containing no strings
4. $(R_1 \cup R_2)$, where R_1 and R_2 are regular expressions,
5. $(R_1 \circ R_2)$, where R_1 and R_2 are regular expressions, or
6. (R_1^*) , where R_1 is a regular expression.

Notation

- In arithmetic $\times$ has precedence over $+$
- In regular expressions *Star* is done first, followed by *concatenation* and finally *union* (unless parenthesis change the usual order)
- R_1 and R_2 are always smaller than R to avoid a circular definition $\rightarrow$ the definition is **inductive**
- $R^+ = RR^* \Rightarrow R^+ \cup \epsilon = R^*$
- $R^k = RRR \dots k \text{ times (concatenation of } k \text{ } R\text{s)}$
- R is the regular expression; $L(R)$ is the language it describes (i.e., the language of R).

Regular Expressions: Examples

- What is the language of $R =$

1. $(0 \cup 1)^*$?

- If $\Sigma = \{0,1\}$ we can write it as Σ^* (the language of all strings possible from the alphabet)
- Σ instead describes the language of all strings of length 1 over the alphabet

2. Σ^*1 ?

1. $(\Sigma\Sigma\Sigma)^*$?

3. $(0\Sigma^*) \cup (\Sigma^*1)$?

2. $(0 \cup \varepsilon)1^*$?

4. 0^*10^* ?

3. $(0 \cup \varepsilon)(1 \cup \varepsilon)$? $= \{\varepsilon, 0, 1, 01\}$

5. $\Sigma^*1\Sigma^*$?

4. $1^*\emptyset$?

Concatenating the empty set yields the empty set

6. $(\Sigma\Sigma)^*$

5. $\emptyset^*$?

Star creates a language composed of all possible strings of the language: if the language is empty, then it can only return the empty string

Regular Expressions: Examples (2)

- What is the language of $R =$

1. $R \cup \emptyset$?

2. $R \circ \varepsilon$?

3. $R \cup \varepsilon$?

4. $R \circ \emptyset$?

Next: We will show that regular expressions and finite automata are equivalent!

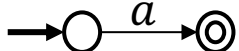
Regular Expressions $\rightarrow$ NFA

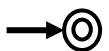
Theorem: If R is a regular expr and $A = L(R)$ then A is regular

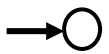
Proof: Convert R to equivalent NFA M :

If R is atomic:

Equivalent M is:

$R = a$ for $a \in \Sigma$ 

$R = \varepsilon$ 

$R = \emptyset$ 

If R is composite:

$R = R_1 \cup R_2$

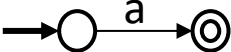
$R = R_1 \circ R_2$

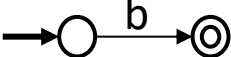
$R = R_1^*$

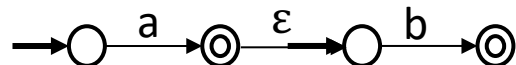
} Use closure constructions

Example:

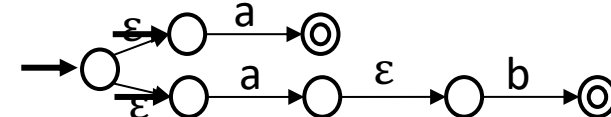
Convert $(a \cup ab)^*$ to equivalent NFA

a : 

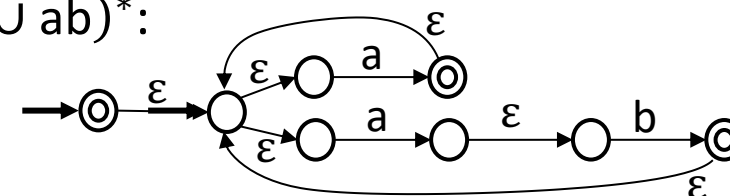
b : 

ab : 

$a \cup ab$:



$(a \cup ab)^*$:



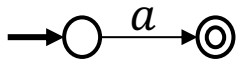
Regular Expressions $\rightarrow$ NFA (Example 2)

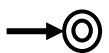
Theorem: If R is a regular expr and $A = L(R)$ then A is regular

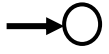
Proof: Convert R to equivalent NFA M :

If R is atomic:

Equivalent M is:

$R = a$ for $a \in \Sigma$ 

$R = \epsilon$ 

$R = \emptyset$ 

If R is composite:

$R = R_1 \cup R_2$

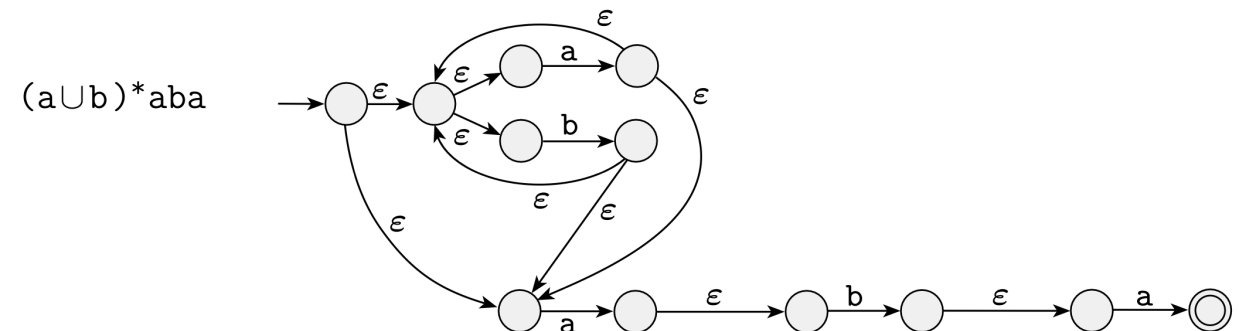
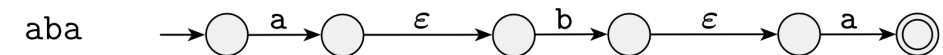
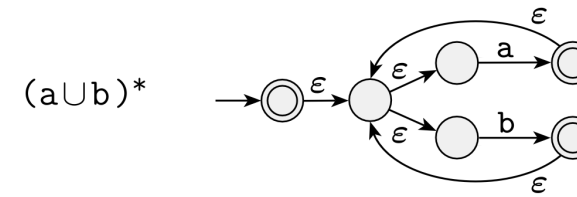
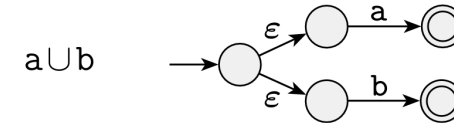
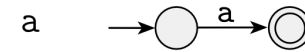
$R = R_1 \circ R_2$

$R = R_1^*$

} Use closure constructions

Example:

Convert $(a \cup b)^*aba$ to equivalent NFA

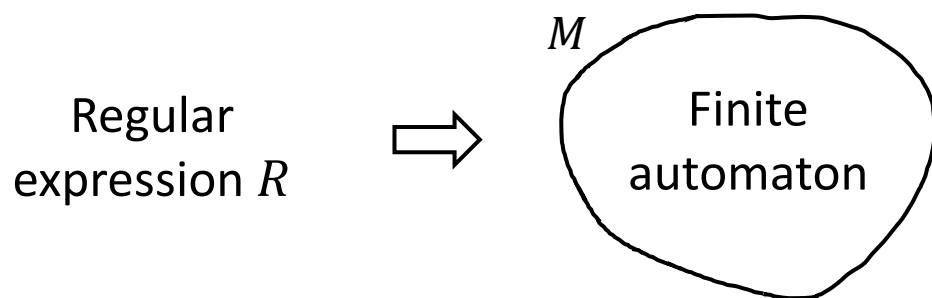


DFAs $\rightarrow$ Regular Expressions

Recall Theorem: If R is a regular expression and $A = L(R)$ then A is regular

Proof: Conversion $R \rightarrow \text{NFA } M \rightarrow \text{DFA } M'$ (last week)
 $\Rightarrow L(R)$ is regular

Are we done with the proof
or do we need to prove
something else?



Are we done? Are regular
expressions equivalent to DFA?

Next Theorem: If A is regular then $A = L(R)$ for some regular expr R

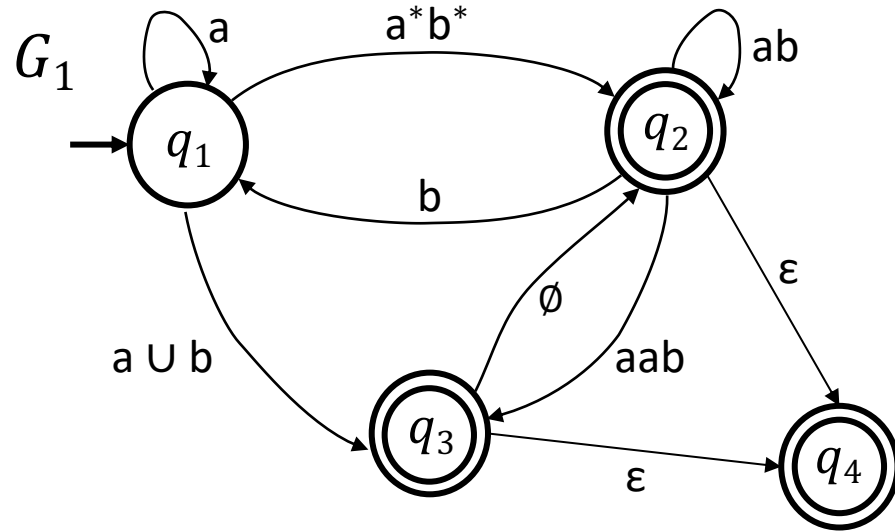
Proof: Give conversion DFA $M \rightarrow R$

WAIT! Need new concept first.

Generalized NFA

Defn: A Generalized Nondeterministic Finite Automaton (GNFA) is similar to an NFA, but allows regular expressions as transition labels

- The GNFA reads blocks of input at a time rather than necessarily one symbol at a time
- It accepts if its processing can cause it to be in an accept state at the end of the input



For convenience we will assume:

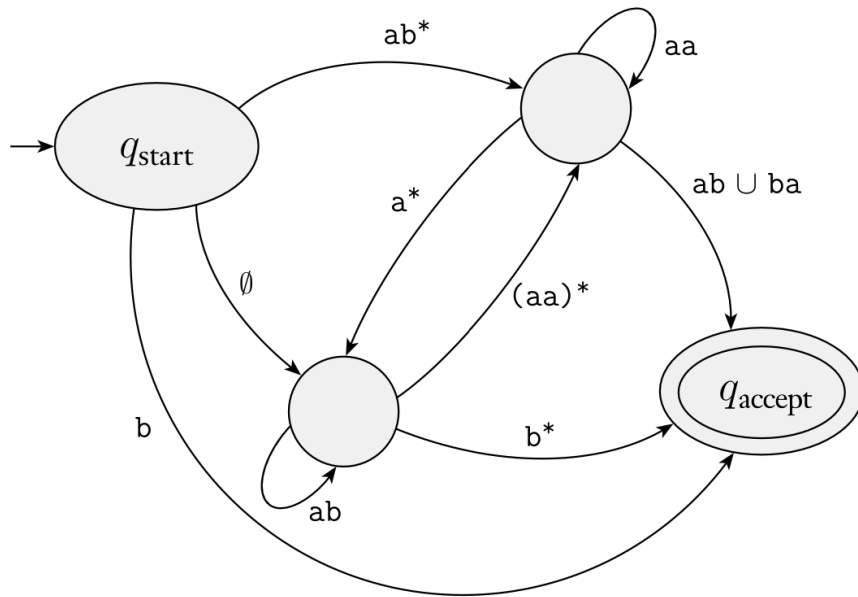
- One accept state, separate from the start state
- One arrow from each state to each state, except
a) only exiting the start state
b) only entering the accept state

We can easily modify a GNFA to have this special form.

Generalized NFA

Defn: A Generalized Nondeterministic Finite Automaton (GNFA) is similar to an NFA, but allows regular expressions as transition labels

- The GNFA reads blocks of input at a time rather than necessarily one symbol at a time
- It accepts if its processing can cause it to be in an accept state at the end of the input



For convenience we will assume:

- One accept state, separate from the start state
- One arrow from each state to each state, except
 - a) only exiting the start state
 - b) only entering the accept state

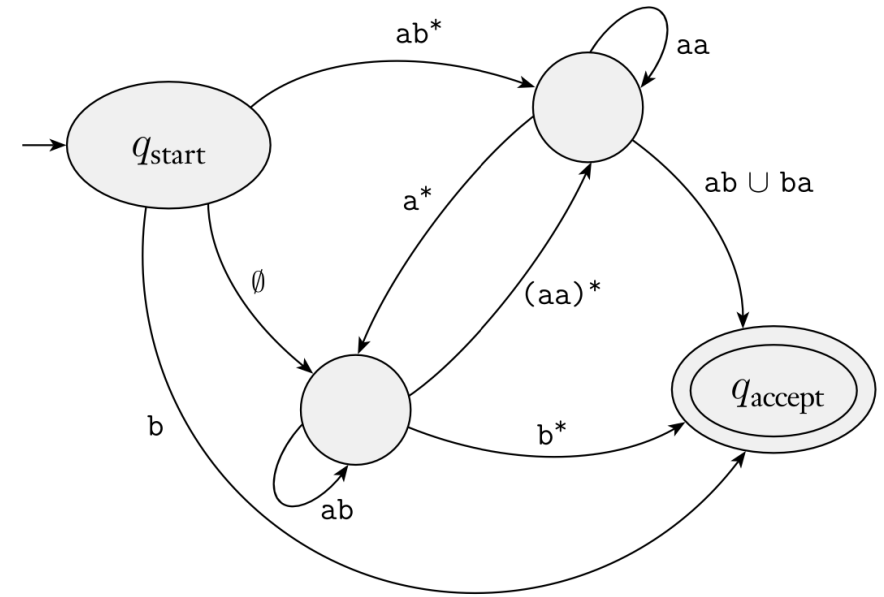
We can easily modify a GNFA to have this special form.

Generalized NFA

A **generalized nondeterministic finite automaton** is a 5-tuple, $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$, where

1. Q is the finite set of states,
2. Σ is the input alphabet,
3. $\delta: (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\}) \rightarrow \mathcal{R}$ is the transition function,
4. q_{start} is the start state, and
5. q_{accept} is the accept state.

The $\mathcal{R}$ symbol is the collection of all regular expressions over Σ



Transition function

- $\delta(q_i, q_j) = R$ means that the arrow from q_i to q_j has the regular expression R on it
- (you have to read R to go from q_i to q_j)
- The domain is $\delta: (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\})$ because the former does not have outgoing arrows and the latter does not have incoming ones

Generalized NFA

A **generalized nondeterministic finite automaton** is a 5-tuple, $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$, where

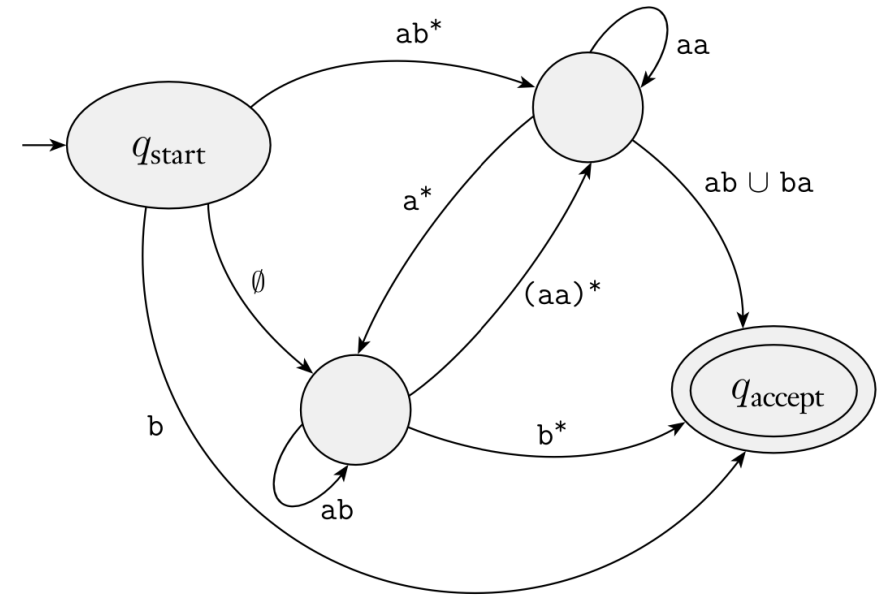
1. Q is the finite set of states,
2. Σ is the input alphabet,
3. $\delta: (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\}) \rightarrow \mathcal{R}$ is the transition function,
4. q_{start} is the start state, and
5. q_{accept} is the accept state.

The $\mathcal{R}$ symbol is the collection of all regular expressions over Σ

Computation

A GNFA accepts a string w in Σ^* if $w = w_1 w_2 \cdots w_k$, where each w_i is in Σ^* and a sequence of states $q_0, q_1, \dots, q_k$ exists such that

1. $q_0 = q_{\text{start}}$ is the start state,
2. $q_k = q_{\text{accept}}$ is the accept state, and
3. for each i , we have $w_i \in L(R_i)$, where $R_i = \delta(q_{i-1}, q_i)$; in other words, R_i is the expression on the arrow from q_{i-1} to q_i .



DFAs $\rightarrow$ Regular Expressions

Next Theorem: If A is regular then $A = L(R)$ for some regular expr R

Proof: Give conversion DFA $M \rightarrow R$

Proof Idea

1. Convert DFA $\rightarrow$ GNFA
2. Convert GNFA $\rightarrow$ Regular expression

DFA → Special GNFA

Algorithm

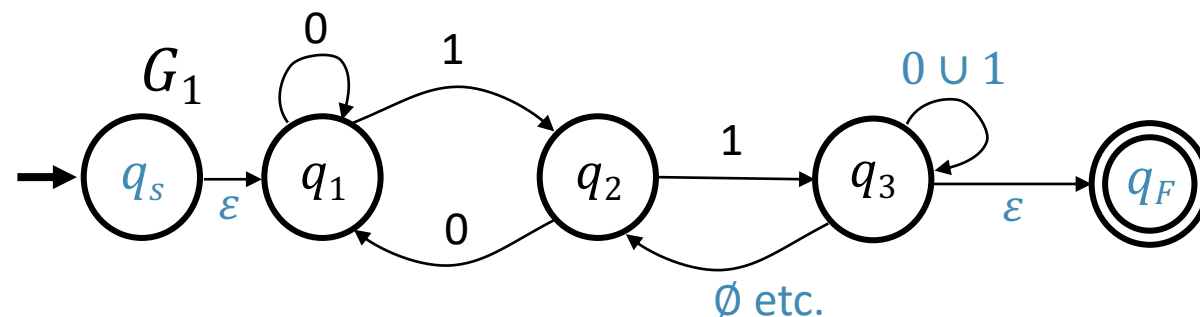
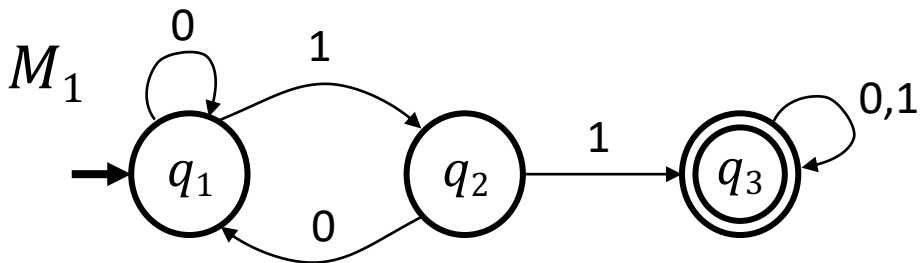
- Add a **new start state** with an ε arrow to the old start state
- Add a **new accept state** with an ε arrow from each accept state
- If any arrows have multiple labels **replace with the union**
- Add arrows labelled $\emptyset$ between states with no arrows

Reminder

For convenience we will assume:

- One accept state, separate from the start state
- One arrow from each state to each state, except
 - a) only exiting the start state
 - b) only entering the accept state

Example



DFAs $\rightarrow$ Regular Expressions

Next Theorem: If A is regular then $A = L(R)$ for some regular expr R

Proof: Give conversion DFA $M \rightarrow R$

Proof Idea

1. Convert DFA $\rightarrow$ GNFA **Done**
2. Convert GNFA $\rightarrow$ Regular expression

GNFA $\rightarrow$ Regular Expressions

Proof Idea

1. Let the GNFA have k states. Then we construct an equivalent GNFA with $k - 1$ states.
2. We repeat the process until we are left with 2 states: start and accept state

Lemma: Every GNFA G has an equivalent regular expression R

Proof: By induction on the number of states k of G

Basis ($k = 2$):

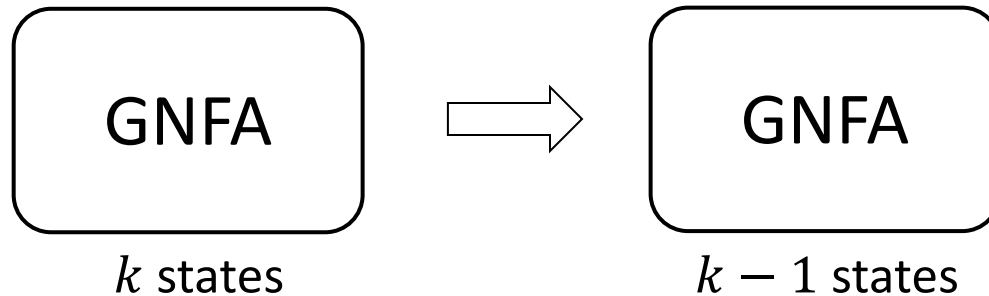
$$G = \rightarrow \bigcirc \xrightarrow{r} \odot$$

Remember: G is in special form

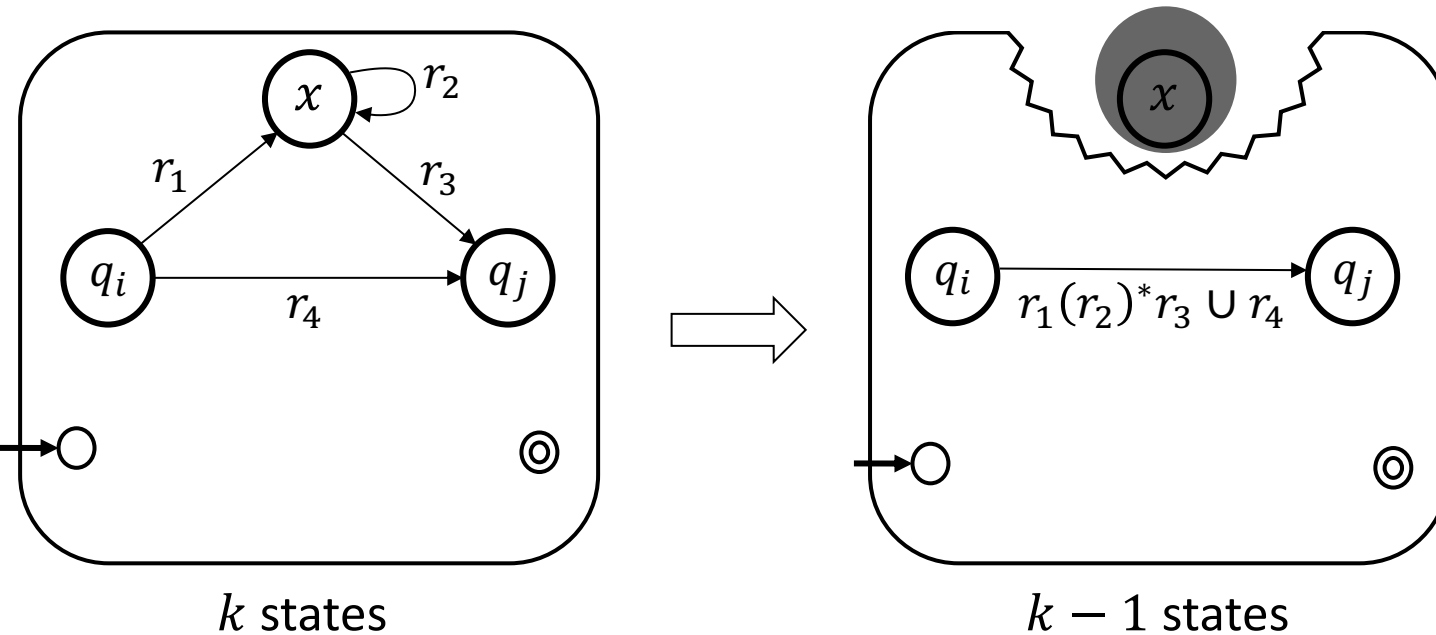
Let $R = r$

Induction step ($k > 2$): Assume Lemma true for $k - 1$ states and prove for k states

IDEA: Convert k -state GNFA to equivalent $(k - 1)$ -state GNFA



k -state GNFA $\rightarrow (k-1)$ -state GNFA



1. Pick any state x except the start and accept states.
2. Remove x .
3. Repair the damage by recovering all paths that went through x .
4. Make the indicated change for each pair of states q_i, q_j .

Thus DFAs and regular expressions are equivalent.

k -state GNFA $\rightarrow (k-1)$ -state GNFA: Algorithm

CONVERT(G):

1. Let k be the number of states of G .
2. If $k = 2$, then G must consist of a start state, an accept state, and a single arrow connecting them and labeled with a regular expression R .
Return the expression R .
3. If $k > 2$, we select any state $q_{\text{rip}} \in Q$ different from q_{start} and q_{accept} and let G' be the GNFA $(Q', \Sigma, \delta', q_{\text{start}}, q_{\text{accept}})$, where

$$Q' = Q - \{q_{\text{rip}}\},$$

and for any $q_i \in Q' - \{q_{\text{accept}}\}$ and any $q_j \in Q' - \{q_{\text{start}}\}$, let

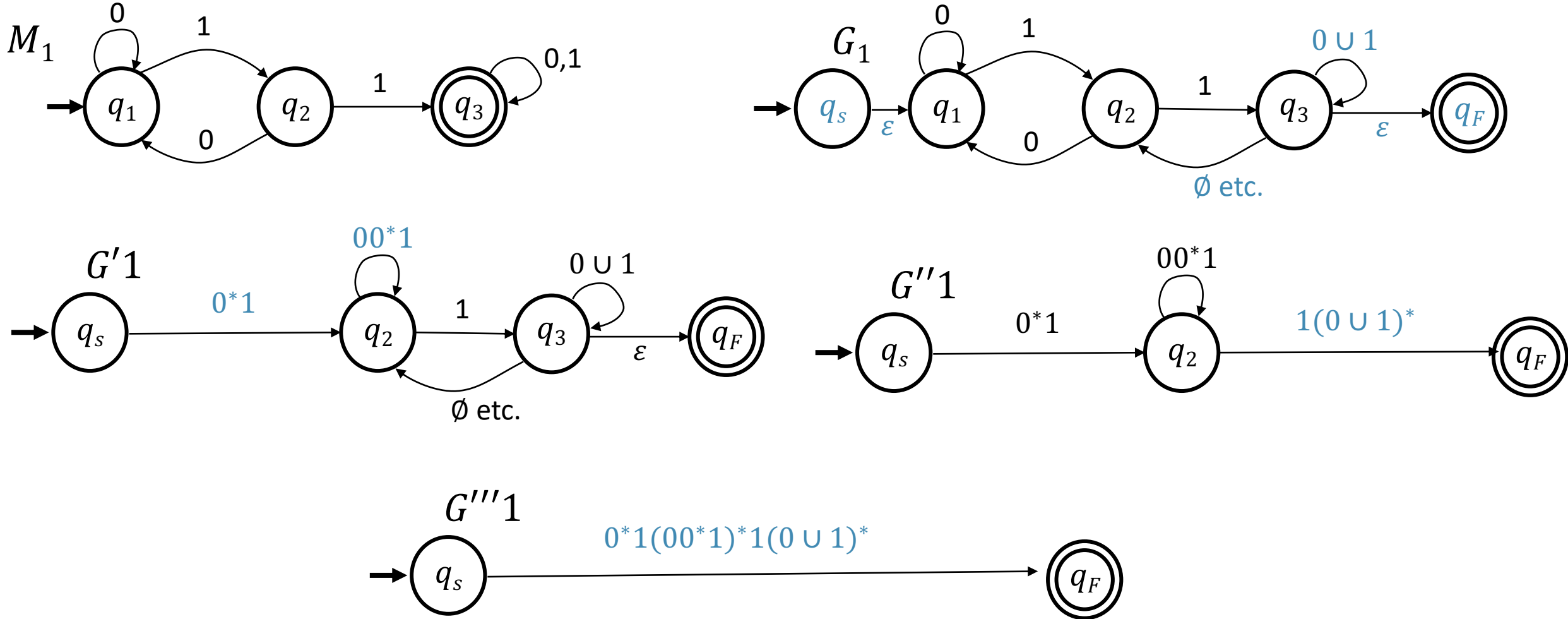
$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4),$$

for $R_1 = \delta(q_i, q_{\text{rip}})$, $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$, $R_3 = \delta(q_{\text{rip}}, q_j)$, and $R_4 = \delta(q_i, q_j)$.

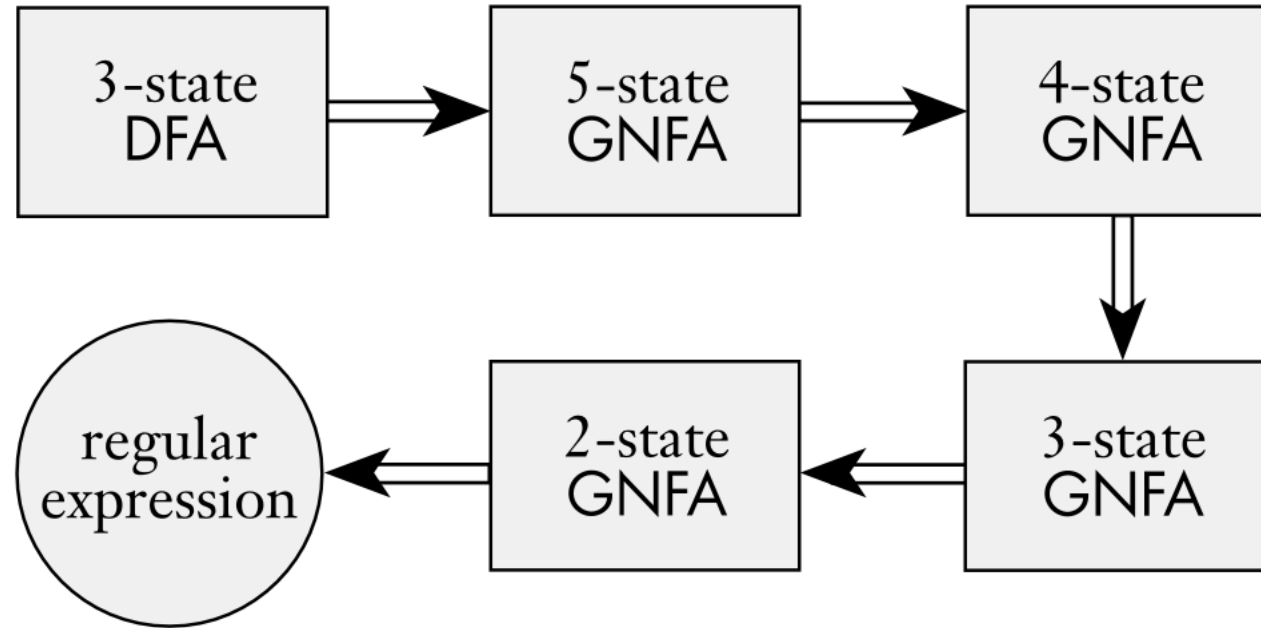
4. Compute CONVERT(G') and return this value.

DFA $\rightarrow$ Special GNFA $\rightarrow$ Reg. expr. : Example

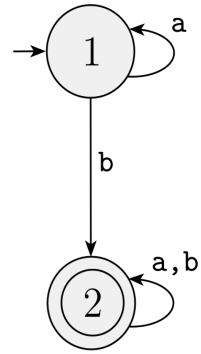
Example



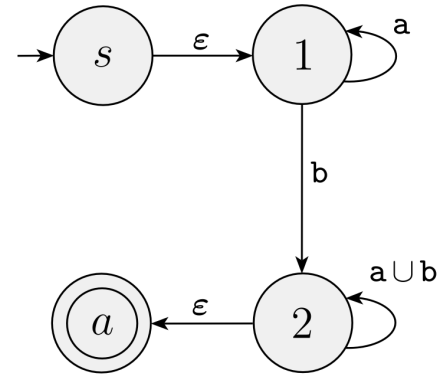
DFA $\Rightarrow$ Regular Expression Process: Example



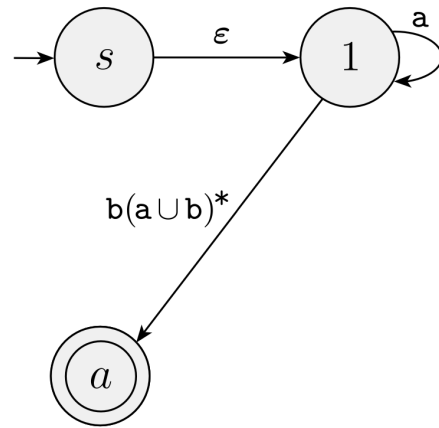
DFAs $\rightarrow$ Regular Expressions: Example



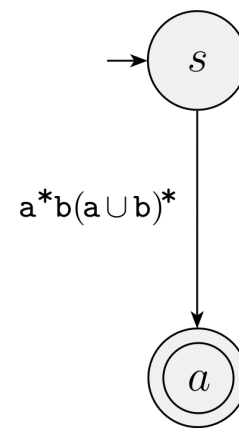
(a)



(b)

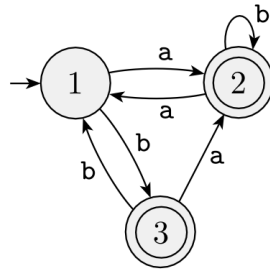


(c)

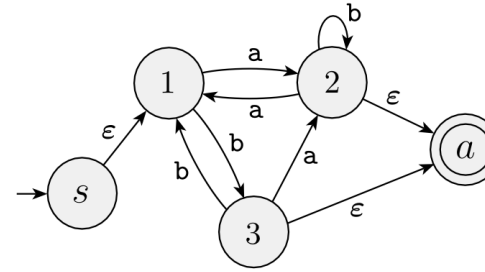


(d)

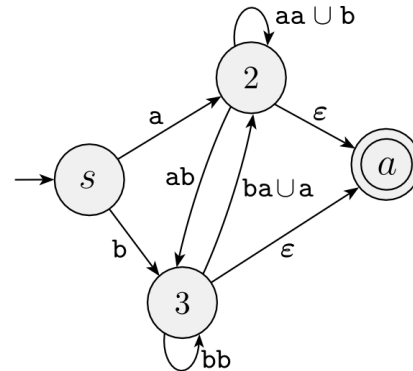
DFAs → Regular Expressions: Example(2)



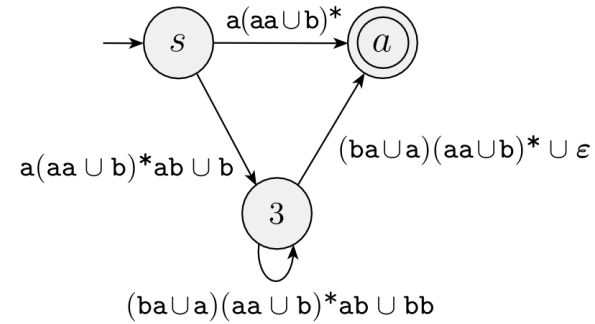
(a)



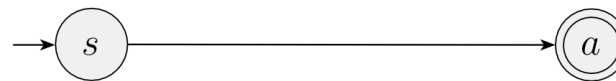
(b)



(c)



(d)



$(a(aa \cup b)^*ab \cup b)((ba \cup a)(aa \cup b)^*ab \cup bb)^*((ba \cup a)(aa \cup b)^* \cup \epsilon) \cup a(aa \cup b)^*$

(e)

Quick review of today

1. Summary: DFAs, NFAs, regular expressions are all equivalent